claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-a5e2080ef9c1b2e67.jsonl`
- SHA-256: `d09f2f03813858f4242302d4d7c0dbc88a07578de185d2459a37d45324c3fa95`
- Source modified: `2026-07-06T19:19:50+00:00`
- Imported at: `2026-07-08T16:00:11+00:00`
- project: `wf_e1b943dd-dac`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T19:18:44.584Z)

```
Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR
(branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F
design doc for a
four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content
safety / Tier 4
deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up
issues F1-F7.
No code changes. Read the ADR:
  git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-
pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design
contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the
codebase and the
other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C
C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem,
Source, Policy),
src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate
should_ingest_source,
delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler:
watched gate -> dedup
-> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track
C (delivery_attempts)
just merged.

Ground rules:
- READ actual files, cite ADR section / file:line. You MAY run
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does
FilterConfig
  ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text
populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies,
not merge-blockers.
  Severity: high (a claim that is false, or a design gap that would break an F-phase / cause
data loss), medium
  (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example,
minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for
implementation.
```

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if
unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).
FINDING (cross_adr): Tier 4 references ADR 0003 but does not propose the amendment/waiver it needs; the phasing table cites a nonexistent "ADR 0003 waiver"
severity medium | section Deep inspection ? Tier 4; phasing F5 ("Dependencies: F1, ADR 0003 waiver"); References
summary: The ADR acknowledges Tier 4 "violates ADR 0003's no-download principle" and F5's dependency column literally says "ADR 0003 waiver," but ADR 0012 neither amends 0003 nor defines a waiver process. ADR 0003 is status Accepted and states ffprobe is available ONLY via an explicit on-demand /inspect command, itself deferred to v2. An accepted ADR should be superseded/amended by a decision, not silently overridden by an opt-in flag in a different ADR.
why_it_matters: "ADR 0003 waiver" is a dangling dependency: there is no artifact that grants it, so F5 has an unsatisfiable prerequisite as written. ADR discipline in this repo is strong (0011 explicitly supersedes 0006, 0010 pattern reuse) ? this ADR breaks that convention. Whoever implements F5 has no defined gate to point at.
evidence: ADR 0003 (Accepted): "True probing via ffprobe is available only through an explicit, on-demand /inspect command ? itself deferred to v2." ADR 0012 Tier 4 + Negative consequence: "Deep inspection violates ADR 0003 and must be explicitly opt-in." F5 row Dependencies = "F1, ADR 0003 waiver." References list only "ADR 0003 ? ... the no-download principle that Tier 4 must opt in to violate" ? a reference, not an amendment. Note 0003 already anticipated exactly this as "a per-channel opt-in for power users in a future version," so an amendment is low-friction ? it just isn't written.
rec: Either add a section to this ADR that formally amends ADR 0003 ("ADR 0003 is amended: ffprobe/download is permitted for content that passes Tiers 1-3 when the owning policy sets deep_inspect=true, subject to per-item/per-account budget") and update 0003's status to "Amended by 0012," or create the waiver as its own tiny ADR and cite it. Replace the vague "ADR 0003 waiver" dependency with the concrete artifact id.

2. assistant (2026-07-06T19:18:48.504Z)

I'll verify this finding by reading the ADR and the referenced ADR 0003.

3. user (2026-07-06T19:18:50.696Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config_json`). The filter is **flat**: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

| Concept | Current state |

```
|---|---|
| `FilterConfig` / `Policy` | One JSON blob per account. Flat duration/resolution/size/exclude
rules. AND semantics. |
| `FilterEngine` | Evaluates all rules; returns `FilterResult(passed, reasons)`. |
| `policies` table | One row per account (is_default=1). `config_json` is the `FilterConfig`
blob. |
| Security / topic / deep-inspect | Nothing. Codec/keyword/MIME filters and ffprobe were
deferred to v2 in the original plan. |
```

What we want

```
A **policy** should be a pipeline of ordered **stages**, each with a
well-defined cost, inputs, and explainable output. The pipeline runs
**after** dedup and **before** delivery. Users should be able to express
topic preferences ("I like AI and philosophy videos"), security rules
("reject links from sketchy domains"), and view **why** a specific item was
accepted or rejected.
```

Decision

Stage model ? four ordered tiers

```
The pipeline is split into four tiers, ordered by cost (cheapest first).
Later stages only run if all earlier stages pass:
```

```
????????????????????????????????????????????????????????????????
?                    Policy Pipeline                           ?
?                                                              ?
?  Tier 1 ? Metadata gates  (always runs, zero I/O)           ?
?     duration, resolution, size, codec, mime, GIF/round       ?
?     ? extends today's FilterEngine                          ?
?                                                               ?
?  Tier 2 ? Topic preferences  (opt-in, metadata-only)         ?
?     source allowlist, keyword match, channel topic tags      ?
?     ? user-expressed interests filter the stream             ?
?                                                              ?
?  Tier 3 ? Content safety  (opt-in, metadata + light I/O)     ?
?     URL/domain reputation, file metadata sanity,             ?
?     caption text scanning                                    ?
?     ? cheap checks; no download                              ?
?                                                              ?
?  Tier 4 ? Deep inspection  (opt-in, heavy I/O)               ?
?     ffprobe, perceptual hash, downloaded content scan        ?
?     ? gated behind explicit user opt-in + budget             ?
????????????????????????????????????????????????????????????????
```

Stage interface ? `PolicyStage`

```
Each stage is a pure function (or async function for I/O stages) that
takes a `PolicyContext` and returns a `StageResult`:
```

```python
@dataclass
class PolicyContext:
    """Immutable snapshot of what the stage can inspect."""
    content_item: ContentItem        # from ADR 0011
    source: Source                   # the source it came from
    policy: Policy                   # the owning policy config

@dataclass
class StageResult:
    passed: bool
```

```
    reason: str                      # human-readable explanation
    stage_name: str                  # e.g. "duration-gate", "url-reputation"
    metadata: dict[str, object] = field(default_factory=dict)  # for explainability UI

class PolicyStage(Protocol):
    """A single gate in the pipeline. Stateless; may be async for I/O stages."""
    async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult` list is the **explanation trace** ? every decision is auditable.

Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json` under a `topics` key:

```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any"  // "any" = match any include topic; "all" = match all
  }
}
```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist` (an explicit list of trusted provider_source_ids).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint` across accounts (already handled by `content_items.dedup_key`; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}`).

Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated behind explicit opt-in** because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true` in `config_json`) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

Policy evaluation flow (post-dedup, pre-delivery)

```python
async def evaluate_pipeline(
    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results  # short-circuit
    return True, results
```

The caller (telethon_client or future Bot API handler) records the
results via `delivery_attempts.last_error` (for failed items) or a new
`policy_decisions` table (for explainability ? deferred to implementation).

Schema ? minimal changes

No schema migration is required for this design. The `policies` table
already holds a `config_json` JSON blob. The pipeline is a code concept:

- `policies.config_json` gains optional `topics`, `allowlist`,
  `security_checks`, `deep_inspect` keys ? all backwards-compatible with
  the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy_decisions` table (deferred) would store the explanation
  trace for debugging: `(content_item_id, policy_id, stage_name, passed,
  reason, metadata_json, evaluated_at)`.

Implementation phasing (future issues)

Phase	Scope	Dependencies
F1 ? Policy engine prototype	`PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler	None
F2 ? Topic preferences	`topics` config parsing, source title/caption keyword matching	F1
F3 ? Source allowlist	`allowlist` config, skip non-allowlisted sources	F1
F4 ? URL/domain reputation	Domain blocklist, caption link extraction	F1
F5 ? Deep inspection	ffprobe/perceptual hash, download budget enforcement	F1, ADR 0003 waiver
F6 ? Policy decisions table	Audit trail storage, explainability UI	F1
F7 ? Explainability endpoint	`/why <content_item_id>` command showing the explanation trace	F6

Each phase is a separate, mergeable PR ? none are bundled here. This ADR
only defines the **design contract** the phases code against.

Consequences

Positive
- Unifies topic, security, and quality gates into a single explainable
  framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for
  items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without
  additional DB lookups at decision time.
- Backwards-compatible: `policies.config_json` already exists; adding
  optional keys requires no migration.
- Each implementation phase is independently shippable behind feature
  flags.

Negative
- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

Alternatives considered

- **Do nothing; keep flat FilterConfig.** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- **Inline each stage in the handler** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- **Use a rules engine / DSL.** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- **Bundle all phases into a single PR.** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

Open questions for the decision owner

1. **`StageResult.metadata` shape** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. **Policy decisions table** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. **Topic matching** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. **Deep inspection budget** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

References

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

4. user (2026-07-06T19:18:51.385Z)

0001-userbot-over-botapi.md
0002-sqlite-over-postgres.md
0003-metadata-first-probing.md
0004-pluggable-delivery-modes.md
0005-per-user-delivery-target.md
0006-content-dedup.md
0007-inline-config-commands.md
0008-bot-api-onboarding-front-door.md

```
0009-selective-channel-scanning.md
0010-identity-and-session-separation.md
0011-provider-neutral-domain-schema.md
```

5. assistant (2026-07-06T19:18:54.985Z)

```
The finding cites ADR 0003 with the filename `0003-metadata-only-filtering.md`, but the actual
file is `0003-metadata-first-probing.md`. Let me read the actual ADR 0003 and verify the
substance of the claim.
```

6. user (2026-07-06T19:18:56.505Z)

ADR 0003: Metadata-first probing (no download in the hot path)

```
- **Status**: Accepted
- **Date**: 2026-07-03
```

Context

```
To filter videos we need properties: duration, resolution (height), file size,
MIME type, and (later) codec/bitrate. There are two ways to obtain them:

1. **Read Telegram message metadata** ? every video `Message` carries a
   `Document` whose attributes include `DocumentAttributeVideo` (duration, `w`,
   `h`) plus `document.size` and `document.mime_type`. This is **free** ? no
   network transfer, no disk.
2. **Download the file and run `ffprobe`** ? gives the *true* codec, bitrate,
   and exact stream dimensions, at the cost of downloading (potentially
   gigabytes per video), needing `ffmpeg` installed, and adding latency to the
   hot path.

For the v1 filters (duration, height ? target, size ? max), the metadata
fields are exactly what we need. Codec/bitrate accuracy is only relevant for
future codec-level filters, which are explicitly out of v1 scope.
```

Decision

```
In the ingestion hot path, build `VideoMeta` **only from Telegram message
metadata** (no download). True probing via `ffprobe` is available only through
an explicit, on-demand `/inspect` command ? itself deferred to v2.
```

Consequences

```
**Positive**
- Ingestion is cheap and fast: no bandwidth, no disk, no `ffmpeg` dependency.
- Latency from "video posted" to "forwarded to Saved Messages" is minimal.
- Container image stays small (no ffmpeg).

**Negative**
- "1080p" is inferred as `height == 1080`. Telegram's metadata is usually
  accurate for dimension, but if a sender mislabels a re-encode, we trust the
  label. Acceptable for v1's quality-bar use case.
- Codec-level rules (e.g. "only H.264, skip AV1") are impossible without
  downloading ? hence their deferral to v2 alongside `/inspect`.
```

Alternatives considered

```
- **Always download + ffprobe**: rejected for the hot path due to bandwidth,
  storage, latency, and the added `ffmpeg` runtime dependency. Could be added
  as a per-channel opt-in for power users in a future version.
- **Metadata first, ffprobe on demand** (the v2 plan): keep the hot path cheap
  and offer an explicit deep-inspect command. This is the chosen evolution
```

path.

7. user (2026-07-06T19:18:58.188Z)

ADR 0011: Provider-neutral domain schema (Track B)

- **Status**: Accepted (Question 1 ? Option 1A; Question 2 ? Option 2A; PR #34)
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #27 (Track B of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0010 (Track A ? `accounts` + `telegram_sessions`, merged in #33)
- **Blocks**: #28 (Track C ? delivery/routing), #29 (Track D ? commands), and indirectly #30 (Track E)

Context

ADR 0010 split `users` into `accounts` (product identity) + `telegram_sessions` (MTProto credential). That unblocked the *identity* dimension. This ADR tackles the **content dimension**: how to represent sources, content items, policies, destinations, routes, and delivery attempts in a provider-neutral way, so that the system can ingest from Telegram today and from WhatsApp (or any future provider) later without another schema rewrite.

What we have today (post-Track-A)

Table	Meaning	Provider-coupling
`accounts`	Product identity (ADR 0010)	provider-neutral (`provider` column)
`telegram_sessions`	Optional MTProto session (ADR 0010)	Telegram-specific by design
`channels`	A Telegram channel/chat an account is a member of	**Telegram-specific** (`channel_id` is a Telegram id)
`videos`	A seen Telegram video message + metadata	**Telegram-specific** (`message_id`, `document_id` are Telegram ids)
`filter_config`	Per-account JSON blob of `FilterConfig`	provider-neutral in shape, but scoped per-account not per-source

The dedup logic (ADR 0006) is **3-tier and Telegram-specific**:
- Tier 0: `(user_id, channel_id, message_id)` ? same Telegram message.
- Tier 1: `(user_id, document_id)` ? same Telegram file forwarded across channels.
- Tier 2: `(user_id, size_bytes, duration_s, height, width)` ? re-upload fingerprint.

What we need

The umbrella (#25) introduces six concepts: `Source`, `ContentItem`, `Policy`, `Destination`, `Route` (`routing_rules`), and `DeliveryAttempt`. This ADR defines the schema for each, the **two open design questions** that gate the schema, and the migration path from the current tables.

The two decisions I need from you

Question 1: `ContentItem.dedup_key` ? how does provider-neutral dedup relate to ADR 0006?

The current 3-tier dedup is Telegram-specific (`document_id` is a Telegram concept). A provider-neutral `ContentItem` needs a dedup story that works across providers. Three options:

Option 1A (recommended): tiered lookup that wraps ADR 0006's tiers

Keep ADR 0006's three tiers but generalize them behind a single `dedup_key` column on `content_items`:

```
content_items.dedup_key TEXT NOT NULL
```

```

The *value* of `dedup_key` is computed by an explicit decision tree at
ingestion time, provider by provider. The tree is ordered strongest-key
first and each branch is **guarded so partial metadata can never collapse
to a degenerate key** (e.g. `tg:fp:0:0:0:0`):

- **Telegram:**
  1. If `document_id` is present ? `dedup_key = "tg:doc:{document_id}"`.
  2. Else if **all four** fingerprint fields (`size_bytes`, `duration_s`,
     `height`, `width`) are present ? `dedup_key = "tg:fp:{size}:{dur}:{h}:{w}"`.
  3. Else ? `dedup_key = "tg:msg:{source_id}:{provider_message_id}"`
     (Tier 0; the source/channel id is required because Telegram
     `message_id` is only unique within a channel, not across sources).
- **Future WhatsApp:** `dedup_key = "wa:msg:{message_id}"` (WhatsApp message
  ids are globally unique per account) or `"wa:sha256:{blob_hash}"` if WA
  exposes a content hash.

The guard in step 2 (all four fields required) is what prevents the
partial-metadata collision the first draft of this ADR allowed. The pure
`compute_dedup_key(provider, *, document_id, source_id, provider_message_id,
size_bytes, duration_s, height, width)` function that implements this tree
gets its own unit tests in the implementation PR, including cases for
partial/null metadata.

=====0006=====

**8. assistant (2026-07-06T19:19:02.772Z)**

Now let me verify the key claims. The finding says F5's dependency cites "ADR 0003 waiver" and
that ADR 0012 neither amends 0003 nor defines a waiver. Both are confirmed by the ADR text I
read. Let me check the ADR-discipline convention claim (that 0011 supersedes 0006) since the
finding cites it as evidence of a strong convention this ADR breaks.

**9. user (2026-07-06T19:19:06.995Z)**

```
===== 0004 =====
3:- **Status**: Accepted
===== 0005 =====
3:- **Status**: Accepted
===== 0006 =====
3:- **Status**: Accepted
===== 0009 =====
3:- **Status**: Accepted
===== 0010 =====
3:- **Status**: Proposed (awaiting decision owner pick between Option A and Option B)
===== 0011 =====
3:- **Status**: Accepted (Question 1 ? Option 1A; Question 2 ? Option 2A; PR #34)
92:- Supersedes ADR 0006 cleanly: the three tiers become *one* column with
307:    status   TEXT NOT NULL DEFAULT 'pending', --
'pending'|'delivered'|'failed'|'retrying'
```

**10. assistant (2026-07-06T19:19:11.219Z)**

Confirmed: ADR 0011 explicitly "Supersedes ADR 0006" ? the repo does have a convention for
formally handling ADR relationships. Let me verify one nuance: does ADR 0006's status reflect
being superseded? And confirm the exact "waiver" and amendment absence in 0012.

**11. user (2026-07-06T19:19:13.653Z)**

```
=== 0012 waiver/amend mentions ===
189:| **F5** ? Deep inspection | ffprobe/perceptual hash, download budget enforcement | F1, ADR
0003 waiver |
```
```

12. assistant (2026-07-06T19:19:39.675Z)

The word "waiver" appears exactly once in ADR 0012 ? in the F5 dependency column ? and nowhere is it defined, and there is no amendment/supersede section. This confirms the core of the finding.

Let me now adversarially test whether the finding overstates the case. The finding claims ADR 0003 says ffprobe is available "ONLY via an explicit on-demand /inspect command, itself deferred to v2." ADR 0003's Decision text confirms this exactly. And its Alternatives section says "Always download + ffprobe... Could be added as a per-channel opt-in for power users in a future version" ? so ADR 0003 does anticipate opt-in deep inspection, meaning an amendment is low-friction (as the finding notes).

Assessing the verdict:

What holds:
- The F5 dependency literally reads "ADR 0003 waiver" (line 189) ? a dangling reference to an artifact that does not exist.
- ADR 0012 defines no waiver process and contains no amendment/supersede section (grep confirms zero occurrences of amend/supersede).
- ADR 0003 is status **Accepted**, and its Decision explicitly restricts ffprobe to the deferred `/inspect` command.
- The repo has a demonstrated convention for formal ADR relationships: ADR 0011 line 92 "Supersedes ADR 0006 cleanly." So overriding an Accepted ADR via an opt-in flag in a different ADR, without amending it, genuinely breaks the established discipline.
- The References section lists ADR 0003 only as a reference ("the no-download principle that Tier 4 must opt in to violate") ? not as an amendment, and does not update 0003's status.

Possible refutations, and why they fail:
- *"It's out of scope for a design doc / Tier 4 is deferred."* The ADR itself introduces the dependency by writing "ADR 0003 waiver" into the phasing table as a concrete prerequisite. Having named it as a gate, it owns the obligation to make it resolvable. Deferring the *implementation* does not excuse a dangling *design dependency*.
- *"Referencing 0003 in the References section is enough."* No ? a reference documents the tension but grants nothing. F5 says it depends on a "waiver," which is an approval artifact, not a citation.
- *Severity high?* No data loss and no false factual claim about the codebase; it's an unsatisfiable/undefined prerequisite and a broken-convention gap. Medium is correct.

One minor note: the finding's evidence block misquotes ADR 0003's filename/title (it references "0003-metadata-only-filtering" / "no-download principle"; the actual file is `0003-metadata-first-probing.md`). But this does not affect the substance ? the quoted Decision text is accurate, and ADR 0012 itself uses the phrase "no-download principle."

The finding genuinely holds against this ADR.

13. user (2026-07-06T19:19:50.802Z)

Structured output provided successfully